CONTRIBUTING

Contributing Guide

Thank you for your interest in contributing to this project! This document provides guidelines and instructions for contributing.

Table of Contents

Code of Conduct

- Be respectful and inclusive
- Focus on constructive feedback
- Help others learn and grow

Getting Started

Prerequisites

- Git
- Docker or Podman
- Bash shell
- Python 3.x (for plugin development)

Fork and Clone

1. Fork the repository on GitHub

2. Clone your fork:

   ```
   git clone git@github.com:YOUR_USERNAME/qBitTorrent.git
   cd qBitTorrent
   ```

3. Add upstream remote:

   ```
   git remote add upstream git@github.com:milos85vasic/
           qBitTorrent.git
   ```

Development Setup

1. Create configuration files:

   ```
   cp .env.example .env
   # Edit .env with your test credentials
   ```

2. Make scripts executable:

   ```
   chmod +x *.sh tests/*.sh
   ```

3. Run tests to verify setup:

   ```
   ./test.sh --all
   ```

How to Contribute

Reporting Issues

1. Check existing issues first
2. Use the issue template
3. Include:
   - Steps to reproduce
   - Expected behavior
   - Actual behavior
   - Environment details (OS, container runtime)

Suggesting Features

1. Open a discussion or issue
2. Describe the feature and use case
3. Explain why it would benefit the project

Submitting Code

1. Create a feature branch:

   ```
   git checkout -b feature/your-feature-name
   ```

2. Make your changes
3. Run tests:

   ```
   ./test.sh --full
   ```

4. Commit your changes
5. Push and create a pull request

Coding Standards

Bash Scripts

- Use `set -euo pipefail` for strict mode
- Add help text with `-h, --help` flags
- Quote all variables
- Use meaningful variable names
- Add color output for user feedback
- Check for command availability

Example:

```bash
#!/bin/bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cd "$SCRIPT_DIR"

RED='\033[0;31m'
GREEN='\033[0;32m'
NC='\033[0m'

print_error() { echo -e "${RED}[ERROR]${NC} $1"; }
```

Python (Plugin Code)

- Follow PEP 8 style guide
- Use type hints where appropriate
- Add docstrings to functions and classes
- Handle exceptions gracefully
- Support environment variables for configuration

Example:

```python
def get_credentials() -> tuple[str, str]:
    """Load credentials from environment or config files.

    Returns:
        Tuple of (username, password)
```

```
    """
    username = os.environ.get("RUTRACKER_USERNAME", "")
    password = os.environ.get("RUTRACKER_PASSWORD", "")
    return username, password
```

Python (Merge Service)

- FastAPI with async handlers
- `aiohttp` for outbound HTTP requests
- `sys.path` setup for test imports using `importlib.util`
- No comments in code (project convention)

YAML/Docker Compose

- Use 2-space indentation
- Include inline comments
- Use descriptive service names
- Document environment variables

Documentation

- Keep README.md updated
- Update USER_MANUAL.md for user-facing changes
- Update AGENTS.md for development changes
- Use clear, concise language

Merge service source files to document: - `download-proxy/src/merge_service/search.py` — search orchestration and tracker parsing - `download-proxy/src/api/routes.py` — API endpoint definitions - `download-proxy/src/api/__init__.py` — API module setup

Testing

Before Committing

Always run tests before committing:

```
# Quick validation
./test.sh

# Full test suite
./test.sh --full

# Or comprehensive tests
./tests/run_tests.sh
```

Test Categories

Test	Command	Description
Quick	`./test.sh`	Basic validation
All	`./test.sh --all`	All validation tests
Plugin	`./test.sh --plugin`	Plugin configuration
Full	`./test.sh --full`	Complete test suite

Merge Service Tests

Run the merge service test suite:

```
python3 -m pytest tests/unit/merge_service/ tests/integration/
        test_merge_api.py -v --import-mode=importlib
```

Adding Tests

1. Add test functions to `tests/run_tests.sh`
2. Follow naming convention: `test_<category>()`
3. Use assertion functions
4. Document the test purpose

Commit Guidelines

Commit Message Format

```
<type>: <subject>

<body (optional)>
```

Types

- `feat`: New feature
- `fix`: Bug fix
- `docs`: Documentation changes
- `style`: Code style changes (formatting)
- `refactor`: Code refactoring
- `test`: Adding or updating tests
- `chore`: Maintenance tasks

Examples

```
feat: add support for custom RuTracker mirrors

- Add RUTRACKER_MIRRORS environment variable
```

```
- Update plugin to use configurable mirrors
- Add documentation for mirror configuration

fix: resolve credential loading from ~/.qbit.env

- Fix path expansion for home directory
- Add fallback to project .env file
- Add test for credential loading
```

Pull Request Process

1. **Create a Branch**
   - Use descriptive branch names
   - One feature/fix per branch
2. **Make Changes**
   - Follow coding standards
   - Add/update tests
   - Update documentation
3. **Test Thoroughly**

   ```
   ./test.sh --full
   ```
4. **Commit Changes**
   - Use meaningful commit messages
   - Reference issues if applicable
5. **Push and Create PR**
   - Push to your fork
   - Create pull request against main
   - Fill out PR template
6. **Code Review**
   - Respond to feedback
   - Make requested changes
   - Keep discussion constructive
7. **Merge**
   - PR will be merged after approval
   - Ensure CI passes

PR Checklist

- [ ]

- [ ] Code follows project style guidelines
- [ ] Tests pass locally
- [ ] Documentation updated
- [ ] Commit messages follow guidelines
- [ ] No sensitive data in commits
- [ ] Branch is up to date with main

Security

Reporting Security Issues

Do NOT open public issues for security vulnerabilities.

Email security issues to the maintainer or use GitHub Security Advisories.

Security Guidelines

- Never commit credentials or secrets
- Use `.env` files (in `.gitignore`)
- Sanitize logs of sensitive data
- Follow principle of least privilege

Questions?

- Open a GitHub Discussion
- Check existing issues
- Review documentation in `docs/`

License

By contributing, you agree that your contributions will be licensed under the Apache License 2.0.

Thank you for contributing!